

From Recursive Chains to Explicit Linearized Fusion

Non-Recursive Multi-ControlNet Composition in COMFYUI for FLUX.1 Workflows

Origin Question, Execution Theory, Architecture, Validation, and Release 2.0 Freeze Record

José Luis Córdova

6 July 2026

Abstract

This paper documents the first JLC non-recursive ControlNet effort: the investigation that began with a practical question—why did adding a second ControlNet Apply stage to a demanding FLUX.1 workflow produce disproportionately longer execution and greater memory pressure than using one?—and evolved into a general architecture for explicit linearized multi-ControlNet fusion in COMFYUI.

The investigation began by separating the user-facing Apply graph from the sampler-facing execution structure. Native Apply nodes do not merely place unrelated controls beside one another. Each new ControlNet is copied, conditioned, and linked through `previous_controlnet`. During sampling, the newest ControlNet recursively requests the output of its predecessor before evaluating and merging its own residuals. This native mechanism is correct and remains useful, especially for one ControlNet. It is also a nested ownership model: model discovery, hook traversal, preparation, cleanup, hint preparation, residual production, and residual merging all follow the linked chain.

The initial hypothesis was that this recursive structure itself explained the observed runtime and memory collapse. The research produced a valuable architecture, but later compatibility and VRAM analysis refined the diagnosis. Native recursion evaluates each child once and is linear in the number of controls; it is not an exponential algorithm. The severe pathological slowdown was traced primarily to forced `-lowvram` behavior interacting with large Flux models, lazy ControlNet hint/VAE preparation, extensive LoRA patching, and destructive partial unload/reload cycles. Nevertheless, recursive merge semantics still impose avoidable tensor-lifetime, allocation, state-inheritance, and reasoning costs. The non-recursive design remains valuable because it makes those costs explicit and controllable.

The final architecture extracts or directly prepares child ControlNets, creates shallow isolated copies, clears every child's `previous_controlnet`, evaluates all active children independently against the same sampler state, and performs explicit weighted streaming fusion. The first owned tensor is cloned and optionally scaled; later residuals are added in place with `dst.add_(value, alpha=weight)`. One ControlNet-compatible wrapper is exposed to the sampler. Unique child models, hooks, preparation, cleanup, and inference-memory requirements are forwarded through current COMFYUI interfaces. A mathematically equivalent single ControlNet at final weight one is routed through the native path rather than through the wrapper.

The architecture supports three workflow interfaces with one fusion theory: Composition linearizes an existing native chain; Orchestrator accepts external ControlNet objects and prepares independent slots; Orchestrator Advanced performs internal model selection, strict preflight, shared-cache reuse, dynamic slot management, and per-run copy isolation. Current validation

on an RTX 4090 Laptop with 16 GB VRAM demonstrated stable heavy FLUX.1 workloads with Union Pro ControlNet, 304 attached LoRA patches, repeated or distinct controls, and as many as four active slots, without model-unload cascades during sampling. Representative 35-step runs completed in approximately 188–202 seconds, while an upper comfort-limit 3.15-megapixel, 50-step run completed in 31:03 sampling time without a full-model unload/reload cascade.

The release 2.0 freeze therefore preserves both parts of the lesson: explicit non-recursive linearization is a useful execution and ownership model, but it must cooperate with rather than replace COMFYUI model management. The community contribution is not the claim that recursion is inherently defective. It is a precise alternative formulation for multi-ControlNet residual fusion, accompanied by a corrected understanding of performance, memory residency, hint preparation, caching, and DynamicVRAM behavior.

Contents

1	Purpose and Contribution	3
2	The Origin Question	3
3	Native ComfyUI ControlNet Semantics	4
3.1	Apply builds a linked chain	4
3.2	Sampling traverses the chain recursively	4
3.3	The native chain is linear, not exponential	5
4	Why Multiple Native Apply Stages Can Feel Disproportionately Expensive	5
4.1	The unavoidable cost	5
4.2	Recursive merge creates extra tensor-lifetime pressure	6
4.3	Strength application and shared-tensor safety complicate ownership	6
4.4	Recursive lifecycle traversal	6
4.5	Lazy hint preparation can coincide at activation boundaries	7
4.6	Model residency is not the sum of staged sizes	7
4.7	Forced low-VRAM mode was the dominant pathological confounder	7
5	Research Methodology and Historical Evolution	8
5.1	Phase 1: native Apply and cache discipline	8
5.2	Phase 2: chain extraction and first Composition release	8
5.3	Phase 3: deterministic streaming fusion and explicit synchronization	8
5.4	Phase 4: Orchestrator interfaces	9
5.5	Phase 5: v2.0.0 compatibility and performance audit	9

6	The Linearized Mathematical Model	9
6.1	Independent operators	9
6.2	User weight and order bias	10
6.3	What is and is not linearized	10
7	Core Implementation	11
7.1	Chain extraction	11
7.2	Shallow detachment	11
7.3	The sampler-facing wrapper	11
7.4	Streaming tensor fusion	12
7.5	Tensor-liveness comparison	12
7.6	Model discovery and identity deduplication	13
7.7	Hooks and lifecycle forwarding	13
7.8	Native single-ControlNet routing	14
8	Three Interfaces, One Theory	14
8.1	JLC ControlNet Composition	14
8.2	JLC ControlNet Orchestrator	14
8.3	JLC ControlNet Orchestrator Advanced	15
8.4	Apply remains intentionally native	15
9	Caching, Preflight, and Per-Run State	15
9.1	Raw bases versus conditioned copies	15
9.2	Hard bypass and strict preflight	16
10	Validation and Performance Evidence	16
10.1	Historical Composition benchmark	16
10.2	Current heavy benchmark: standard heavy Flux base	17
10.3	Heavier base model: FLUX_SRPO_bf16	17
10.4	Upper comfort-limit stress test	17
10.5	Orchestrator stress tests	18
10.6	Correctness and determinism	18
11	Corrected Interpretation of the Performance Question	19

11.1 What the initial hypothesis got right	19
11.2 What the later investigation corrected	19
11.3 Why non-recursive composition remains worthwhile	19
12 DynamicVRAM and Operational Guidance	20
12.1 Validated configuration	20
12.2 Operational rule	20
13 MultiGPU Scope	20
14 Release 2.0 Technical Freeze Statement	21
15 Community-Level Theory Summary	21
16 Recommended Future Work	22
A Reference Pseudocode	22
B Regression Checklist	23
C Complexity and Memory Notes	24

1 Purpose and Contribution

This companion paper records the theory and engineering history behind the JLC ControlNet Composition and Orchestrator family for FLUX.1-oriented COMFYUI workflows. It is intentionally broader than user documentation. The goal is to explain:

- the original performance question;
- the exact native recursive execution structure that the question exposed;
- the distinction between recursive control flow, computational complexity, tensor lifetime, and model residency;
- the evolution from chain flattening to an explicit reusable fusion core;
- the reasons for shallow copy, detached linkage, owned-output cloning, and in-place later accumulation;
- the relationship among Apply, Composition, Orchestrator, and Orchestrator Advanced;
- the corrected diagnosis of pathological low-VRAM behavior;
- the validation evidence and remaining limits; and
- the release 2.0 technical freeze contract.

The paper focuses on FLUX.1 because that was the workload that exposed the problem and supplied the most demanding validation environment. The core abstraction is not limited to one ControlNet architecture. It composes any compatible COMFYUI ControlNet-like objects whose `get_control()` outputs follow the native residual dictionary/list structure.

2 The Origin Question

The research began with a deceptively simple observation:

Why does a workflow with two ControlNet Apply stages take so much longer and create so much more memory pressure than the same workflow with one?

A naive expectation is approximately linear scaling. If one ControlNet requires one additional model evaluation per denoising step, two should require roughly two. Some overhead is expected from a second hint image, a second model, and a second set of residual tensors. The observed behavior, however, could be far more severe and variable. In demanding FLUX.1 workflows, the second or third control could coincide with long pauses, aggressive offload activity, VAE or AutoencodingEngine preparation, low-VRAM weight-patch handling, and occasionally catastrophic runtime growth.

The user-facing graph did not explain the behavior. Two Apply nodes look like two ordinary stages. The decisive insight was that the graph is transformed into a linked sampler object before sampling begins.

3 Native ComfyUI ControlNet Semantics

3.1 Apply builds a linked chain

A native-style Apply node copies the supplied base ControlNet, applies per-run hint and timing state, retrieves any ControlNet already attached to the conditioning row, and links the new copy to it:

Listing 1: Native-style chain construction preserved by JLC Apply.

```
d = conditioning_metadata.copy()
prev_cnet = d.get("control", None)

c_net = (
    control_net.copy()
    .set_cond_hint(
        control_hint,
        strength,
        (start_percent, end_percent),
        vae,
    )
)
c_net.set_previous_controlnet(prev_cnet)
d["control"] = c_net
```

After three Apply stages, the conditioning metadata points to the newest object, which points to the previous object, which points to the oldest:

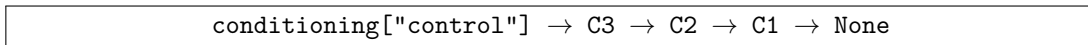


Figure 1: Native `previous_controlnet` linkage after three Apply stages.

The Apply stage is therefore a chain builder, not a fusion stage.

3.2 Sampling traverses the chain recursively

The core ControlNet evaluation pattern is structurally equivalent to:

Listing 2: Simplified native recursive evaluation.

```
def get_control(self, x_noisy, t, cond, batched_number, transformer_options):
    control_prev = None
    if self.previous_controlnet is not None:
        control_prev = self.previous_controlnet.get_control(
            x_noisy, t, cond, batched_number, transformer_options
        )

    control_current = self.control_model(...)
    return self.control_merge(control_current, control_prev)
```

The oldest child is reached first. Its residuals return to the next child, whose own model executes and merges with the previous result. The aggregate then returns to the next level.

For three controls, the precise conceptual flow is not literally a composition of latent transforms such as $C_3(C_2(C_1(x)))$. The controls all receive the same sampler state x ; what is recursively nested is residual production and merge ownership. A more accurate recurrence is:

$$R_1 = M_1(C_1(x), 0),$$

$$R_2 = M_2(C_2(x), R_1),$$

$$R_3 = M_3(C_3(x), R_2),$$

where $C_i(x)$ is the current child output and M_i is the native strength/application/merge operation at that chain level.

3.3 The native chain is linear, not exponential

A key correction from the later investigation is that native recursion does not automatically mean repeated exponential evaluation. For n active ControlNets, the ordinary chain evaluates approximately n ControlNet models per denoising call. Its model-forward complexity is $O(n)$, just like a sequential flat loop.

The useful question is therefore not:

Does recursion cause the same ControlNet to execute exponentially many times?

It is:

What additional ownership, allocation, lifetime, model-management, preparation, and scheduling behavior follows from the nested chain representation?

That distinction is central to the final theory.

4 Why Multiple Native Apply Stages Can Feel Disproportionately Expensive

4.1 The unavoidable cost

Two active ControlNets do require two side-model evaluations. No composition wrapper can make two distinct control conditions cost the same as one without approximating or skipping work. The unavoidable costs include:

- each child model’s forward pass;
- each child’s ControlNet residual tensors;

- hint preprocessing and possible latent-format conversion;
- VAE or AutoencodingEngine encoding where required;
- model weights or dynamically staged model material; and
- any independent activation range and per-step control logic.

A correct comparison must begin from this baseline.

4.2 Recursive merge creates extra tensor-lifetime pressure

At each recursive level, the previous aggregate must remain alive while the current child runs and while a new merge result is produced. The native merge path historically used out-of-place addition for overlapping entries:

Listing 3: Conceptual native merge allocation.

```
if current_value is None:
    current_value = previous_value
else:
    current_value = previous_value + current_value
```

At one merge point, memory can temporarily include:

- the previous aggregate $R_{1\dots i-1}$;
- the current child output $C_i(x)$; and
- the newly allocated aggregate $R_{1\dots i}$.

Python recursion itself is not the significant memory cost. The large residual tensors and their liveness are.

4.3 Strength application and shared-tensor safety complicate ownership

Native ControlNet output dictionaries can contain lists with `None` entries, shared tensor references, different lengths, and architecture-specific keys. The native merge path must avoid applying strength more than once to shared tensors and cannot always assume in-place ownership. This favors conservative merge behavior.

A flat composer can define a stronger ownership contract: never mutate child output; clone only when the composite first takes ownership; then mutate only composite-owned storage.

4.4 Recursive lifecycle traversal

The linked structure is traversed not only by `get_control()` but also by lifecycle and discovery methods such as:

- `get_models()`;
- `get_extra_hooks()`;

- `pre_run()`;
- `cleanup()`; and
- inference-memory reporting.

A well-behaved native chain handles this correctly. However, custom wrappers must implement the complete surface. Omitting deduplication, forwarding, or cleanup can produce duplicated model registrations, stale hint state, incorrect memory estimates, or subtle incompatibility after upstream changes.

4.5 Lazy hint preparation can coincide at activation boundaries

ControlNet hints are commonly prepared lazily, inside the first active `get_control()` call at the required latent size. For Flux ControlNets, this can invoke a VAE or AutoencodingEngine and restore previously active model residency afterward.

When several controls become active at the same denoising boundary, one step can therefore include:

- multiple hint resizes;
- one or more VAE encodes;
- latent-format conversion;
- ControlNet model preparation; and
- model-manager decisions under already high VRAM pressure.

This is valid native behavior, but it explains why the transition from one to two controls can look nonlinear in a constrained system.

4.6 Model residency is not the sum of staged sizes

DynamicVRAM logs can report, for example, a 6.3 GB ControlNet and a 22.7 GB Flux model as staged at the same time on a 16 GB GPU. These logical staged sizes are not simultaneous physical occupancy. They describe model material prepared for dynamic loading and patching. The model manager keeps only working portions resident.

Interpreting staged sizes as a simple sum leads to incorrect conclusions. The relevant evidence is actual unload/reload behavior, step timing, low-VRAM patch count, and stability.

4.7 Forced low-VRAM mode was the dominant pathological confounder

The later v2.0.0 audit established that the most dramatic runtime collapse was primarily caused by launching COMFYUI with forced `-lowvram`. Under that mode, hint-preparation boundaries and large LoRA-patched Flux models could trigger destructive partial unload/reload cycles and deferred weight-patch handling.

This produced highly variable behavior ranging from roughly twelve minutes to extreme multi-hour cases in pathological tests. Those runs cannot be used as clean proof that recursive ControlNet chaining itself is catastrophically inefficient.

The corrected conclusion is:

The original question correctly exposed a difficult interaction among multiple controls, residual ownership, hint preparation, and model residency. The non-recursive architecture solved real ownership and predictability problems. The most extreme slowdown, however, was dominated by forced low-VRAM behavior rather than by recursive asymptotic complexity.

5 Research Methodology and Historical Evolution

5.1 Phase 1: native Apply and cache discipline

The earliest public JLC node preserved native Apply semantics while improving graph ergonomics. Apply Advanced added optional internal model selection, hard bypass for disabled or zero-strength stages, session-level cache reuse, and per-run copies. This phase established two principles that later became essential:

1. a cached base model must remain unconditioned; and
2. hint, strength, schedule, and chain linkage belong to a per-run copy.

5.2 Phase 2: chain extraction and first Composition release

The first Composition node treated the completed native chain as input data. It traversed `previous_controlnet`, reversed the resulting newest-to-oldest traversal into oldest-to-newest order, shallow-copied each member, cleared the links, and exposed one wrapper to the sampler.

The initial public performance study used three controls at 1024 by 1536 pixels on an RTX 4090 Laptop: OpenPose, HED, and Depth with Union Pro, a 35-step Flux workflow, and 16-bit major components. Historical release notes reported approximately 2.5-times lower average runtime and approximately five-times lower runtime variance than the recursive baseline in that controlled test. This was evidence that the flattened implementation was practical and promising, not a universal complexity theorem.

5.3 Phase 3: deterministic streaming fusion and explicit synchronization

A subsequent release refined the implementation into streaming tensor fusion and added explicit CUDA synchronization after each child. Synchronization was useful during early investigation because it made child completion, ownership, and release behavior easier to observe and reduced ambiguity from asynchronous execution.

It was later recognized as too expensive for the normal path. In the v2 core, synchronization is disabled by default and retained only as an environment-controlled diagnostic:

```
$env:JLC_CONTROLNET_SYNC_AFTER_CHILD = "1"
```

5.4 Phase 4: Orchestrator interfaces

Composition required users to build a native chain first and then flatten it. The Orchestrator family removed that construction step:

- the external Orchestrator accepts ControlNet objects from loaders;
- the Advanced Orchestrator selects registered files internally;
- each active slot creates an isolated conditioned copy; and
- all prepared children are passed directly to the same fusion core.

This separated model sourcing and user interface from execution mathematics.

5.5 Phase 5: v2.0.0 compatibility and performance audit

The current freeze followed a full audit against modern COMFYUI sampler, model-management, hook, inference-memory, and MultiGPU-related interfaces. The pass corrected real defects and removed misleading assumptions:

- shared core rather than divergent Composition and Orchestrator math;
- strict shallow-copy policy and removal of `deepcopy`;
- unique model discovery by object identity;
- full lifecycle and hook forwarding;
- sequential scratch-memory reporting as a maximum, not a sum;
- native single-ControlNet routing when mathematically equivalent;
- default removal of mandatory per-child CUDA synchronization;
- strict preflight before internal model loading;
- shared cache ownership of raw bases only;
- current single-device compatibility shunts; and
- explicit refusal to claim real MultiGPU cloning.

A severe slowdown that triggered the audit was ultimately traced mainly to forced `-lowvram`. A separate Composition import defect was real but had been introduced only during the final refactor immediately before testing. Other lifecycle and cache issues found in the audit were also real and were corrected.

6 The Linearized Mathematical Model

6.1 Independent operators

Let active child ControlNets be indexed by i . Every child receives the same sampler state:

$$x = (x_{\text{noisy}}, t, \text{cond}, b, \text{transformer options}).$$

Each child produces a residual structure:

$$C_i(x) = \{K \mapsto [r_{i,K,0}, r_{i,K,1}, \dots]\},$$

where keys commonly include `input`, `middle`, and `output`, but the composer does not assume that every child provides every key or every position.

The explicit fused output is:

$$R(x) = \sum_{i=0}^{n-1} W_i C_i(x).$$

The sum is interpreted elementwise over matching keys and list positions, ignoring `None` values and creating positions when first introduced by a later child.

6.2 User weight and order bias

Nodes that expose the order-bias parameter α use:

$$W_i = w_i \alpha^i,$$

where w_i is the slot or extracted-chain weight and i is the original zero-based position.

Important semantics follow:

- $\alpha = 1$ gives neutral positional weighting;
- $0 < \alpha < 1$ favors earlier positions;
- $\alpha > 1$ favors later positions;
- negative w_i values are supported;
- Orchestrators may permit negative α , producing alternating signs; and
- skipped slots retain their original exponent rather than renumbering later slots.

At $\alpha = 1$, the mathematical sum is order-invariant. Floating-point accumulation order can still create tiny numerical differences, so practical “invariance” means no systematic semantic order bias rather than bitwise commutativity across arbitrary reorderings.

6.3 What is and is not linearized

The composer does not linearize the internal transformer computation of a ControlNet. Each C_i remains a full nonlinear neural network. The linearization occurs at the residual-output aggregation boundary:

$$\text{nonlinear child models} \quad \longrightarrow \quad \text{linear weighted residual sum.}$$

The architecture changes how outputs are owned and merged, not how each ControlNet interprets its hint or conditioning.

7 Core Implementation

7.1 Chain extraction

Composition begins with the ControlNet attached to conditioning and traverses the native links. Object identity is tracked to avoid an infinite loop if a malformed cycle exists:

Listing 4: Oldest-to-newest native-chain extraction.

```
def extract_controlnet_chain(cnet):
    chain = []
    visited = set()
    while cnet is not None and id(cnet) not in visited:
        chain.append(cnet)
        visited.add(id(cnet))
        cnet = getattr(cnet, "previous_controlnet", None)
    chain.reverse()
    return chain
```

Oldest-to-newest order matters because chain weight row 1 and α^0 correspond to the oldest member.

7.2 Shallow detachment

Every extracted child is isolated with `copy.copy()`, not `deepcopy`:

Listing 5: Shallow detachment.

```
def make_detached_chain(chain):
    detached = []
    for cnet in chain:
        c_copy = copy.copy(cnet)
        c_copy.previous_controlnet = None
        c_copy.multigpu_clones = {}
        detached.append(c_copy)
    return detached
```

The copy is intentionally shallow because the large underlying model patcher and weights should remain shared. Only mutable per-object chain linkage and compatibility bookkeeping need isolation.

Deep copying would be both semantically wrong and operationally dangerous: it could duplicate model structures, break patcher ownership, consume large amounts of RAM/VRAM, and bypass the host model manager.

7.3 The sampler-facing wrapper

The sampler receives one `JLC_ComposedControlNet` object. It owns lists of detached children and final weights and presents a ControlNet-compatible surface:

Listing 6: Essential wrapper state.

```
class JLC_ComposedControlNet:
    def __init__(self, controlnets, weights):
        self.controlnets = list(controlnets)
        self.weights = list(weights)
        self.previous_controlnet = None
        self.extra_hooks = None
        self.multigpu_clones = {}
```

The wrapper is not a neural model. It is a lifecycle and residual-aggregation owner.

7.4 Streaming tensor fusion

The first child output that contributes to a given key and list position is cloned. That clone becomes composite-owned storage:

Listing 7: First-output ownership.

```
owned = value.clone()
if weight != 1.0:
    owned.mul_(weight)
combined[key][index] = owned
```

Later outputs are accumulated in place:

Listing 8: Later weighted accumulation.

```
dst = combined[key][index]
if dst is None:
    owned = value.clone()
    if weight != 1.0:
        owned.mul_(weight)
    combined[key][index] = owned
else:
    dst.add_(value, alpha=weight)
```

This implements the central ownership rule:

Never mutate child-owned output. Clone only when the composite first takes ownership.
Mutate only composite-owned storage afterward.

The implementation tolerates child `None` returns, `None` entries, keys introduced by later children, differing list lengths, zero weights, and absent children.

7.5 Tensor-liveness comparison

For one residual slot, a recursive out-of-place merge can temporarily require:

previous aggregate + current child output + new aggregate.

Streaming in-place composition requires approximately:

owned accumulator + current child output.

This is not a universal closed-form VRAM reduction because model weights, attention scratch, allocator behavior, and architecture-specific output sizes dominate many workflows. It is, however, a concrete reduction in avoidable merge allocation and output lifetime.

7.6 Model discovery and identity deduplication

The sampler sees one wrapper, but COMFYUI must still manage every real child model. The wrapper aggregates child model lists and deduplicates by object identity:

Listing 9: Unique child model discovery.

```
def get_models(self):
    models = []
    for cnet in self.controlnets:
        models += cnet.get_models()
    return unique_by_identity(models)
```

This is particularly important when several slots use shallow copies of the same cached base model. There may be multiple conditioned control objects but only one underlying model patcher.

7.7 Hooks and lifecycle forwarding

The wrapper forwards:

- `get_extra_hooks()` to every child;
- `pre_run()` to every child;
- `cleanup()` once per unique child object;
- `get_models_only_self()` to the deduplicated model list;
- `copy()` as a shallow wrapper copy; and
- `get_instance_for_device()` as a single-device compatibility shunt.

The wrapper reports the maximum child temporary inference-memory requirement, not the sum:

$$M_{\text{scratch}}^{\text{wrapper}} = \max_i M_{\text{scratch}}^{(i)}.$$

Because children execute sequentially, summing scratch estimates would falsely claim all temporary workspaces must coexist. Child model weights remain separately discoverable through `get_models()`.

7.8 Native single-ControlNet routing

The routing helper classifies the effective active set:

Table 1: Routing policy.

Effective active controls	Route
None	Conditioning passes through unchanged.
One at final weight 1.0	Native single-ControlNet path.
One at a non-unit final weight	Composed wrapper, preserving the declared scalar exactly.
Two or more	Composed wrapper.

The native single path is not a reluctant fallback. It is the preferred design whenever the composed and native mathematics are equivalent. It avoids wrapper overhead and preserves the most direct COMFYUI behavior.

If incoming conditioning already contains a prior ControlNet, native single routing reconnects it with `set_previous_controlnet(prev_cnet)`.

8 Three Interfaces, One Theory

8.1 JLC ControlNet Composition

Composition is the original cornerstone and modular interface. Users build an ordinary native chain with Apply nodes, then place Composition before the sampler:

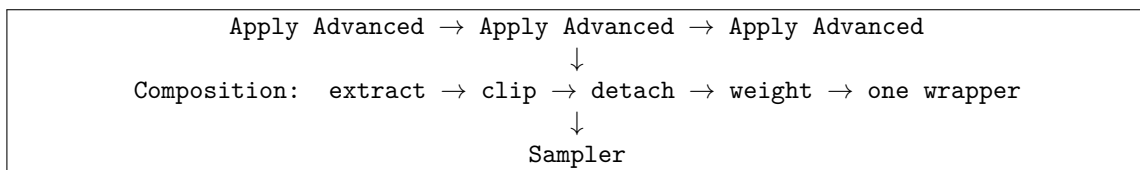


Figure 2: Modular chain-building followed by explicit linearization.

Composition supports dynamic weight rows, chain clipping, order bias, diagnostics, and replacement reuse between matching positive and negative conditioning rows.

This interface is not deprecated by Orchestrator Advanced. It remains ideal when explicit Apply stages, pass-through wiring, external sourcing, or separate graph-level control are desirable.

8.2 JLC ControlNet Orchestrator

The external-input Orchestrator receives ControlNet objects from visible upstream loaders. For each active slot, it:

1. resolves the ControlNet object;

2. receives the hint image and timing/strength settings;
3. creates an isolated per-run copy;
4. applies native `set_cond_hint()`; and
5. delegates children and weights to the shared core.

It is the correct interface for custom loaders, nonstandard locations, shared workflow branches, dynamically modified ControlNets, or specialized device/precision policy.

8.3 JLC ControlNet Orchestrator Advanced

Orchestrator Advanced is the integrated interface for ordinary registered ControlNet files. It adds:

- dynamic visible slots;
- internal file selectors;
- `DISABLED` and `SHARE_PREVIOUS` semantics;
- strict whole-node preflight before any load;
- shared family-scoped LRU cache reuse;
- per-slot image, strength, range, and weight;
- bounded cache capacity; and
- the same final fusion core as Composition and external Orchestrator.

The shared cache stores only raw base ControlNet objects. It never stores a hint, activation range, previous chain, or other per-run state. Every active slot creates a fresh conditioned copy.

8.4 Apply remains intentionally native

The JLC Apply nodes preserve COMFYUI chain semantics. They are useful on their own and are the natural chain-building stages for modular Composition workflows. Apply Advanced does not silently perform non-recursive fusion; Composition is the explicit linearization boundary.

Keeping these concepts separate prevents a dangerous ambiguity where an Apply node sometimes constructs a native chain and sometimes flattens it without the graph making the change visible.

9 Caching, Preflight, and Per-Run State

9.1 Raw bases versus conditioned copies

A ControlNet object can contain both heavy shared model ownership and mutable per-run state. The final cache discipline separates them:

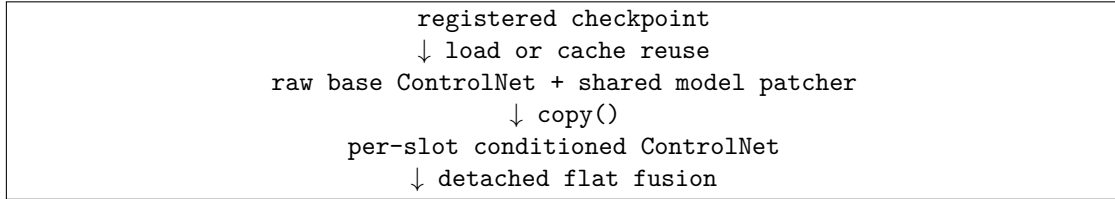


Figure 3: Shared base ownership and per-run mutable state.

This permits repeated use of one model with different images, strengths, or schedules without contaminating the cached object.

9.2 Hard bypass and strict preflight

Inactive stages should not load models. Apply Advanced returns before model resolution when disabled or at zero strength. Orchestrator Advanced determines the active slot set before touching the cache.

A slot is inactive when its selector is disabled, strength is zero, weight is zero, or its range is empty. An otherwise active slot with no image raises a clear error before model loading. `SHARE_PREVIOUS` must resolve to an earlier named model.

This order matters on a 16 GB system: validation failure should not leave one or more large ControlNets resident.

10 Validation and Performance Evidence

10.1 Historical Composition benchmark

The first public Composition benchmark used:

Table 2: Historical three-ControlNet benchmark configuration.

Item	Configuration
Base workflow	FLUX.1-dev-oriented, 16-bit major components
ControlNet	FLUX.1-dev-ControlNet-Union-PRO
Controls	OpenPose, HED, Depth
Resolution	1024×1536
Sampling	35 steps, CFG approximately 2.1
Hardware	RTX 4090 Laptop, 16 GB VRAM
Method	Repeated randomized runs and repeated seeds
Historical observation	Approximately 2.5-times faster average runtime and five-times lower variance than the tested recursive baseline

The result motivated continued development. The later low-VRAM investigation means it should be read as workflow evidence, not as proof that flat fusion is always 2.5-times faster than native

recursion.

10.2 Current heavy benchmark: standard heavy Flux base

A v2-era modular Apply Advanced $\rightarrow$ Composition benchmark produced:

Table 3: Representative validated 35-step benchmark.

Metric	Result
ControlNetFlux staged size	6,297 MB
Flux staged size	11,350 MB
Attached LoRA patches	304
Sampling steps	35
Sampling time	approximately 188 s
Total prompt time	197.97 s
Full-model unload events during sampling	0

10.3 Heavier base model: FLUX_SRPO_bf16

Keeping the toolchain and benchmark otherwise consistent while using one of the heaviest routinely used FLUX.1 bases produced:

Table 4: Representative heavy-base benchmark.

Metric	Result
ControlNetFlux staged size	6,297 MB
Flux staged size	22,700 MB
Attached LoRA patches	304
Sampling steps	35
Sampling time	approximately 202 s
Total prompt time	224.63 s
Full-model unload events during sampling	0

Although the logical staged Flux size roughly doubled, total prompt time increased by only about 26.7 seconds. AutoencodingEngine preparation occurred afterward without a destructive model unload cascade.

10.4 Upper comfort-limit stress test

The practical upper comfort-limit test for the classic modular Composition workflow used:

- resolution 1448×2176 , approximately 3.15 megapixels;
- 50 sampling steps;

- FLUX_SRPO_bf16;
- Union Pro ControlNet, approximately 6,297 MB staged;
- Flux model, approximately 22,700 MB staged;
- 304 attached LoRA patches;
- multiple ControlNet uses, with one disabled;
- HED, DSINE, and DWPose preprocessing; and
- normal VRAM mode with DynamicVRAM on the 16 GB RTX 4090 Laptop.

The result was 31:03 sampling time, 31:15 total prompt time, approximately 37.28 seconds per iteration, no full-model unload during sampling, and a 159 MB AutoencodingEngine staged afterward. The generated result was described as spectacular and photo-like.

This test represents a performance cliff or practical comfort boundary, not a recommended routine resolution. Its significance is that the system remained architecturally stable rather than falling into the earlier forced-low-VRAM unload/reload pathology.

10.5 Orchestrator stress tests

Additional Orchestrator Advanced runs exercised:

- repeated and distinct ControlNet models;
- more LoRAs;
- as many as four ControlNet uses;
- overlapping and non-overlapping activation ranges; and
- the same 16 GB GPU.

Performance remained steady at comparable speeds. This supports the claim that the shared core and lifecycle integration are robust across interfaces, not merely in one lucky Composition graph.

10.6 Correctness and determinism

Validation covered more than runtime:

- pose, edge, and depth structure remained effective;
- no systematic quality degradation was observed relative to the tested recursive baseline;
- slot-order bias behaved as declared through α ;
- inactive slots did not load or execute;
- single-ControlNet native routing preserved ordinary behavior;
- repeated model names shared cached bases but retained distinct per-run hint state;
- positive and negative conditioning rows reused matching wrapper objects where safe; and
- cleanup and repeated execution remained stable.

11 Corrected Interpretation of the Performance Question

11.1 What the initial hypothesis got right

The initial focus on recursive chaining correctly identified several real issues:

- the execution structure was hidden behind simple Apply nodes;
- residual merging involved nested ownership and avoidable allocations;
- state inheritance through `previous_controlnet` made independent reasoning difficult;
- model/hook/lifecycle traversal required careful compatibility work;
- explicit flat fusion made weights and order bias inspectable; and
- a single sampler-facing wrapper enabled one coherent composition policy.

11.2 What the later investigation corrected

The later audit established that:

- native recursion is still $O(n)$ in child forward evaluations;
- multiple ControlNets necessarily cost more than one;
- staged size is not physical simultaneous residency;
- lazy hint preparation can make one denoising boundary unusually expensive;
- extensive LoRA patching changes offload behavior; and
- forced `-lowvram` was the main cause of the catastrophic multi-hour test state.

The corrected theory is stronger because it no longer depends on blaming recursion for every symptom.

11.3 Why non-recursive composition remains worthwhile

Even after the correction, the architecture offers durable advantages:

1. **Explicit mathematics.** The final residual is visibly $\sum W_i C_i(x)$.
2. **Independent child state.** No child inherits another child's chain during evaluation.
3. **Controlled tensor ownership.** Only the first owned result is cloned; later merges are in place.
4. **Predictable model discovery.** Unique child models are exposed once.
5. **Correct scratch reporting.** Sequential temporary memory is represented by a maximum.
6. **Workflow flexibility.** Existing chains can be flattened, or children can be prepared directly.
7. **Native preservation.** One mathematically equivalent control remains native.
8. **Easier diagnostics.** Child order, weights, routing, cache use, and activation decisions can be logged explicitly.

12 DynamicVRAM and Operational Guidance

12.1 Validated configuration

The release 2.0 baseline was validated with:

Table 5: Compatibility baseline.

Component	Validated configuration
COMFYUI tag relation	v0.25.0-34-g2a61015
COMFYUI commit	2a610155821d670a2d8047e654e5fce96b790eb5
Commit date	22 June 2026
Frontend	1.45.19
Python	3.10.11
PyTorch	2.9.1+cu130
CUDA reported by PyTorch	13.0
GPU	NVIDIA RTX 4090 Laptop, 16 GB VRAM
Runtime mode	Normal VRAM; DynamicVRAM enabled; no forced <code>-lowvram</code>

12.2 Operational rule

For the validated 16 GB Flux workflow:

Allow COMFYUI and DynamicVRAM to manage model residency normally. Do not force `-lowvram` unless a separate controlled test demonstrates that it is necessary and beneficial.

The JLC core should not impose a competing global residency policy. Its responsibilities are accurate model discovery, safe copies, correct temporary-memory reporting, and clean lifecycle forwarding.

13 MultiGPU Scope

The composed wrapper includes the interface attributes expected by current COMFYUI code:

```
self.previous_controlnet = None
self.multigpu_clones = {}
```

These are compatibility shunts for single-device execution, not real MultiGPU support. `get_instance_for_device()` returns the current wrapper, and `deepclone_multigpu()` raises a clear error.

Real MultiGPU support would require independently owned child model patchers on each device, device-aware hint state, explicit scheduling, transfer policy, cleanup, and hardware validation. It is intentionally outside the freeze because it cannot be responsibly advertised without a suitable test system.

14 Release 2.0 Technical Freeze Statement

Frozen milestone — As of 6 July 2026, the following architecture is accepted as the frozen JLC FLUX.1 non-recursive ControlNet baseline:

1. Native Apply nodes continue to build ordinary `previous_controlnet` chains.
2. Composition may extract a native chain in oldest-to-newest order, protect against cycles, clip it according to visible slots, and shallow-detach selected members.
3. Orchestrator interfaces may prepare independent child copies directly without constructing an intermediate native chain.
4. Cached base ControlNets remain unconditioned; all hint, strength, schedule, VAE, and linkage state belongs to per-run copies.
5. `deepcopy` is prohibited from normal composition and orchestration.
6. Every child participating in non-recursive fusion has `previous_controlnet=None` and cleared inherited MultiGPU clone bookkeeping.
7. The sampler sees one `JLC_ComposedControlNet` object.
8. Children are evaluated sequentially against the same sampler state.
9. The first contributing output tensor is cloned when the composite takes ownership.
10. Later outputs are accumulated into owned storage with weighted in-place addition.
11. Child output storage is never mutated by the composite.
12. Unique child model patchers are exposed through `get_models()`.
13. Hooks, preparation, cleanup, copying, and inference-memory behavior are forwarded through the complete current compatibility surface.
14. Sequential scratch requirements are reported as the largest child requirement, not the sum.
15. No effective control yields pass-through; one effective control at final weight one yields native execution; weighted singles and multiple controls yield composed execution.
16. Mandatory per-child CUDA synchronization is disabled in the normal path and remains available only as a diagnostic option.
17. Orchestrator Advanced validates complete active-slot wiring before loading models and uses a shared cache of raw bases only.
18. Real MultiGPU cloning is explicitly unsupported.
19. The validated runtime recommendation is normal VRAM mode, optionally with DynamicVRAM, without forced `-lowvram`.

No subsequent feature should alter these invariants without preserving this state in version control and rerunning the regression matrix.

15 Community-Level Theory Summary

The complete theory can be reduced to five statements:

1. **Apply is not composition.** Apply constructs a linked native ControlNet chain.

2. **Native recursion is linear but nested.** It evaluates each child once, yet residual ownership, merge allocation, state inheritance, and lifecycle traversal are nested.
3. **Flat fusion is an alternative ownership model.** Independent child outputs can be summed explicitly after detaching their links.
4. **Performance is a system property.** Model forwards, hint preparation, VAE use, LoRA patching, DynamicVRAM, and launch mode can dominate the difference between architectures.
5. **The host model manager must remain authoritative.** A custom composer should expose models and requirements accurately rather than forcing residency.

The contribution is therefore both an algorithm and a diagnostic framework. It offers an explicit residual algebra while preventing that algebra from being confused with every observed VRAM or runtime symptom.

16 Recommended Future Work

The freeze leaves several useful directions:

1. Build a standardized benchmark harness that records per-step time, peak allocated and reserved VRAM, model load/unload events, hint-preparation boundaries, and output similarity across native and composed execution.
2. Add optional tensor-lifetime instrumentation to quantify merge-allocation savings directly.
3. Explore safe reuse of identical hint encodes within one run when model, image, VAE, resolution, and schedule permit it.
4. Test broader ControlNet architectures, including models that introduce uncommon output keys or list layouts.
5. Design real MultiGPU composition only when appropriate hardware is available.
6. Compare the FLUX.1 residual-composition theory with the newer block-injection strategy developed independently for FLUX.2.

A Reference Pseudocode

Listing 10: Complete conceptual non-recursive fusion.

```
def compose_controlnets(children, weights, x, t, cond, batch, options):
    combined = None

    for child, weight in zip(children, weights):
        if child is None or weight == 0:
            continue

        # Every child is detached: child.previous_controlnet is None.
        output = child.get_control(x, t, cond, batch, options)
        if output is None:
            continue

    if combined is None:
```



```

combined = {}

for key, values in output.items():
    dst_values = combined.setdefault(key, [None] * len(values))
    if len(dst_values) < len(values):
        dst_values.extend([None] * (len(values) - len(dst_values)))

    for index, value in enumerate(values):
        if value is None:
            continue

        if dst_values[index] is None:
            owned = value.clone()
            if weight != 1.0:
                owned.mul_(weight)
            dst_values[index] = owned
        else:
            dst_values[index].add_(value, alpha=weight)

del output

return combined

```

B Regression Checklist

A release derived from this freeze should verify:

1. One active ControlNet at final weight one routes through native execution.
2. One active ControlNet at non-unit final weight routes through the composed wrapper and preserves the exact scalar.
3. Two or more active children have `previous_controlnet=None` during evaluation.
4. Extracted chain order is oldest to newest.
5. Skipped slots retain original α exponents.
6. Negative weights behave as declared.
7. First output tensors are cloned; later outputs are accumulated in place.
8. Child-owned tensors are not modified.
9. Keys and list positions introduced by later children are preserved.
10. Repeated shallow copies of one base model yield one unique model patcher in discovery.
11. Positive and negative conditioning rows reuse compatible wrapper objects where intended.
12. Disabled and zero-strength Apply Advanced stages do not load models.
13. Orchestrator Advanced missing-image errors occur before loading.
14. `SHARE_PREVIOUS` resolves only after a prior named model.
15. Per-child CUDA synchronization is off by default.
16. Normal VRAM mode completes the standard heavy benchmark without a full-model unload cascade.
17. Forced `-lowvram` is not used for release validation.

18. MultiGPU attempts fail clearly rather than silently mixing ownership.

C Complexity and Memory Notes

Let n be the number of active ControlNets, F_i the cost of child i 's neural forward pass, O_i its residual output size, and S_i its temporary inference scratch.

Both native sequential recursion and flat sequential composition require approximately:

$$T_{\text{forward}} \approx \sum_{i=1}^n F_i.$$

The flat architecture does not change this fundamental sum.

Its primary theoretical memory advantage is merge ownership. A simplified recursive out-of-place merge can require the prior aggregate, current output, and new aggregate at once. Streaming composition retains one owned aggregate and one current output:

$$M_{\text{flat merge}} \approx M_{\text{accumulator}} + \max_i O_i,$$

rather than repeatedly allocating a new full aggregate. Sequential scratch should be budgeted as:

$$M_{\text{scratch}} \approx \max_i S_i,$$

while model weight residency remains a model-manager question involving all unique child patchers and the base diffusion model.

References

- [1] J. L. Córdova, *JLC ComfyUI Nodes*, GitHub repository. <https://github.com/Damkohler/jlc-comfyui-nodes>
- [2] J. L. Córdova, *ControlNet Composition and Orchestration*, JLC ComfyUI Nodes documentation. <https://github.com/Damkohler/jlc-comfyui-nodes/blob/main/docs/controlnet-composition.md>
- [3] J. L. Córdova, *JLC ComfyUI Nodes v1.1.3 — ControlNet Composition (Non-Recursive Aggregation)*, GitHub release, 7 April 2026. <https://github.com/Damkohler/jlc-comfyui-nodes/releases/tag/v1.1.3>
- [4] J. L. Córdova, *Release v1.2.0 — Deterministic Streaming ControlNet Composition*, GitHub release, 10 April 2026. <https://github.com/Damkohler/jlc-comfyui-nodes/releases/tag/v1.2.0>
- [5] J. L. Córdova, *v1.3.0 — JLC ControlNet Orchestrator (Base & Advanced)*, GitHub release, 17 April 2026. <https://github.com/Damkohler/jlc-comfyui-nodes/releases/tag/v1.3.0>

- [6] ComfyUI contributors, *ComfyUI ControlNet implementation*, GitHub. <https://github.com/comfyanonymous/ComfyUI/blob/master/comfy/controlnet.py>
- [7] ComfyUI contributors, *ComfyUI source tree, pinned validation commit 2a610155821d670a2d8047e654e5fce96b790eb5*, GitHub. <https://github.com/comfyanonymous/ComfyUI/tree/2a610155821d670a2d8047e654e5fce96b790eb5>
- [8] XLabs-AI and ComfyUI contributors, *Flux ControlNet architecture as integrated in ComfyUI*, GitHub source history. <https://github.com/XLabs-AI/x-flux>